

Agenda

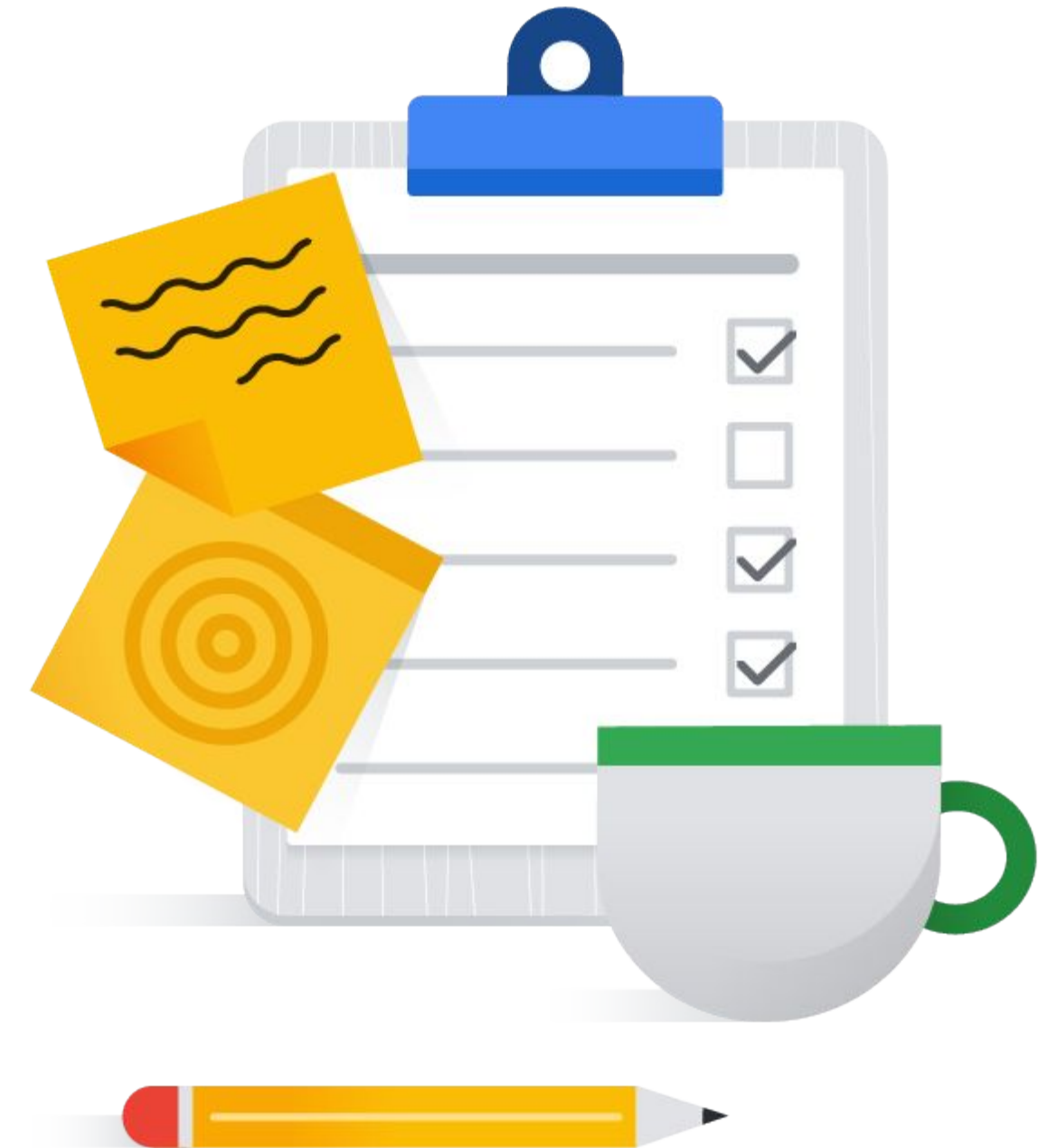
- 01 Introducing Agent Development Kit
- 02 Develop agents with ADK
- 03 Build multi-agent systems with ADK
- 04 Deploy ADK agents to Agent Engine
- 05** Evaluate ADK agent systems



Evaluate and test ADK agent systems

Module objectives

01	Compare different types of ADK evaluation.
02	Explain the differences between test files and evalset files.
03	Discover how to run ADK evaluations using different methods.



Types of ADK evaluation



Types of ADK evaluation

Evaluating the final response

Assessing the quality, relevance, and correctness of the agent's final output

Evaluating trajectory and tool use

Analyzing the steps an agent takes to reach a solution (choice of tools, strategies, and efficiency of approach)

Types of ADK evaluation

Evaluating the final response

- Compares a response to a reference response in your evaluation dataset
- Configure an expected `response_match_score` to consider a test passed or failed
- ROUGE metric calculates the similarity between final and expected responses



Types of ADK evaluation



Evaluating trajectory and tool use

Compares expected to actual values of:

- `determine_intent`
- `use_tool`
- `review_results`
- `report_generation`

Configure an expected
`tool_trajectory_avg_score` to
consider a test passed or failed

Ground-truth-based trajectory evaluations

01

Exact match: Requires a perfect match to the ideal trajectory

02

In-order match: Requires the correct actions in the correct order; allows for extra actions

03

Any-order match: Requires the correct actions in any order; allows for extra actions

04

Precision: Measures the relevance or correctness of predicted actions

05

Recall: Measures how many essential actions are captured in the prediction

06

Single-tool use: Checks for the inclusion of a specific action

ADK evaluation methods



ADK evaluation methods



Using a test file



Using an evalset file

ADK evaluation methods



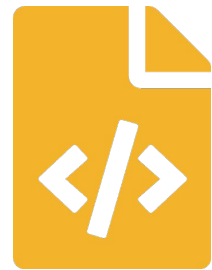
Using a test file

Involves creating individual test files, each representing a single, simple agent-model interaction (session)



Using an evalset file

ADK evaluation methods



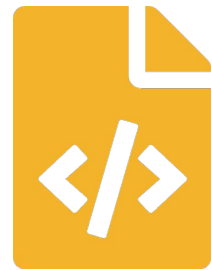
Using a test file



Using an evalset file

```
[{
  "query": "hi",
  "expected_tool_use": [],
  "reference": "Hello! What can I do for you?\n",
},
{
  "query": "roll a die for me",
  "expected_tool_use": [
    {
      "tool_name": "roll_die",
      "tool_input": {
        "sides": 6
      }
    }
  ]
},
{
  "query": "what's the time now?",
  "expected_tool_use": [],
  "reference": "I'm sorry, I cannot access real-time
information, including the current time. My capabilities are
limited to rolling dice and checking prime numbers.\n"
}]
```

ADK evaluation methods



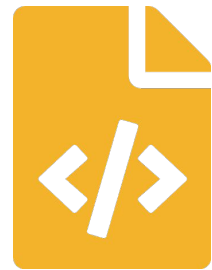
Using a test file



Using an evalset file

Utilizes a dedicated dataset - an "evalset" - for evaluating agent-model interactions

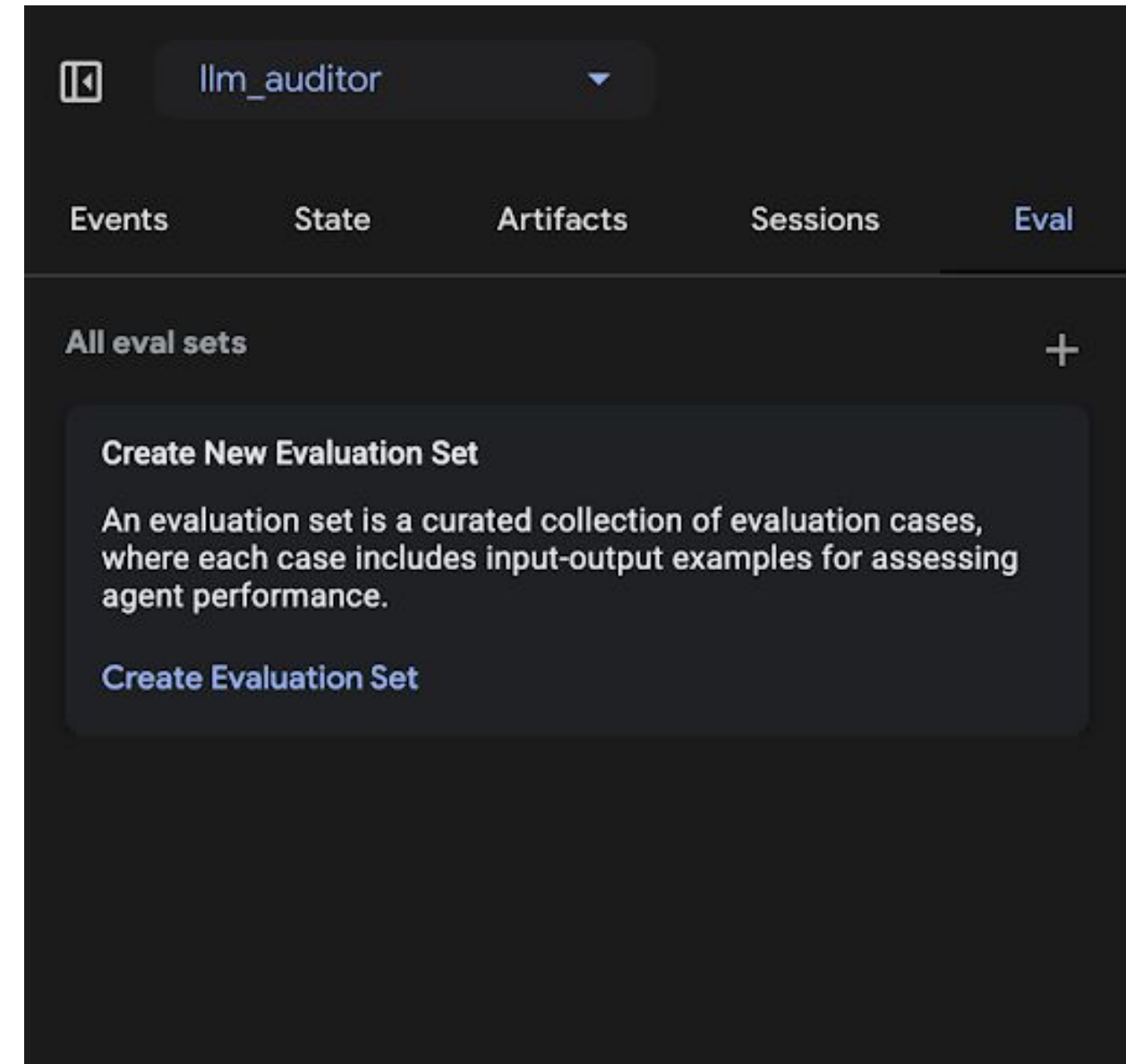
ADK evaluation methods



Using a test file



Using an evalset file



- Create an evaluation set file.
- Add a current session as an eval case to your file.
- Run evaluations with a new version of your agent.

Run evaluations with ADK



How to run evaluations with ADK

Web UI (adk web)

Programmatically
(pytest)

Command Line
Interface (adk eval)

How to run evaluations with ADK

Web UI (adk web)

Evaluate agents
interactively through a
web-based interface

Programmatically (pytest)

Command Line Interface (adk eval)

How to run evaluations with ADK

Web UI (adk web)

Evaluate agents interactively through a web-based interface

Programmatically (pytest)

Integrate evaluation into your testing pipeline using pytest and test files

Command Line Interface (adk eval)

How to run evaluations with ADK

Web UI (adk web)

Evaluate agents interactively through a web-based interface

Programmatically (pytest)

Integrate evaluation into your testing pipeline using pytest and test files

Command Line Interface (adk eval)

Run evaluations on an existing evaluation set file directly from the command line

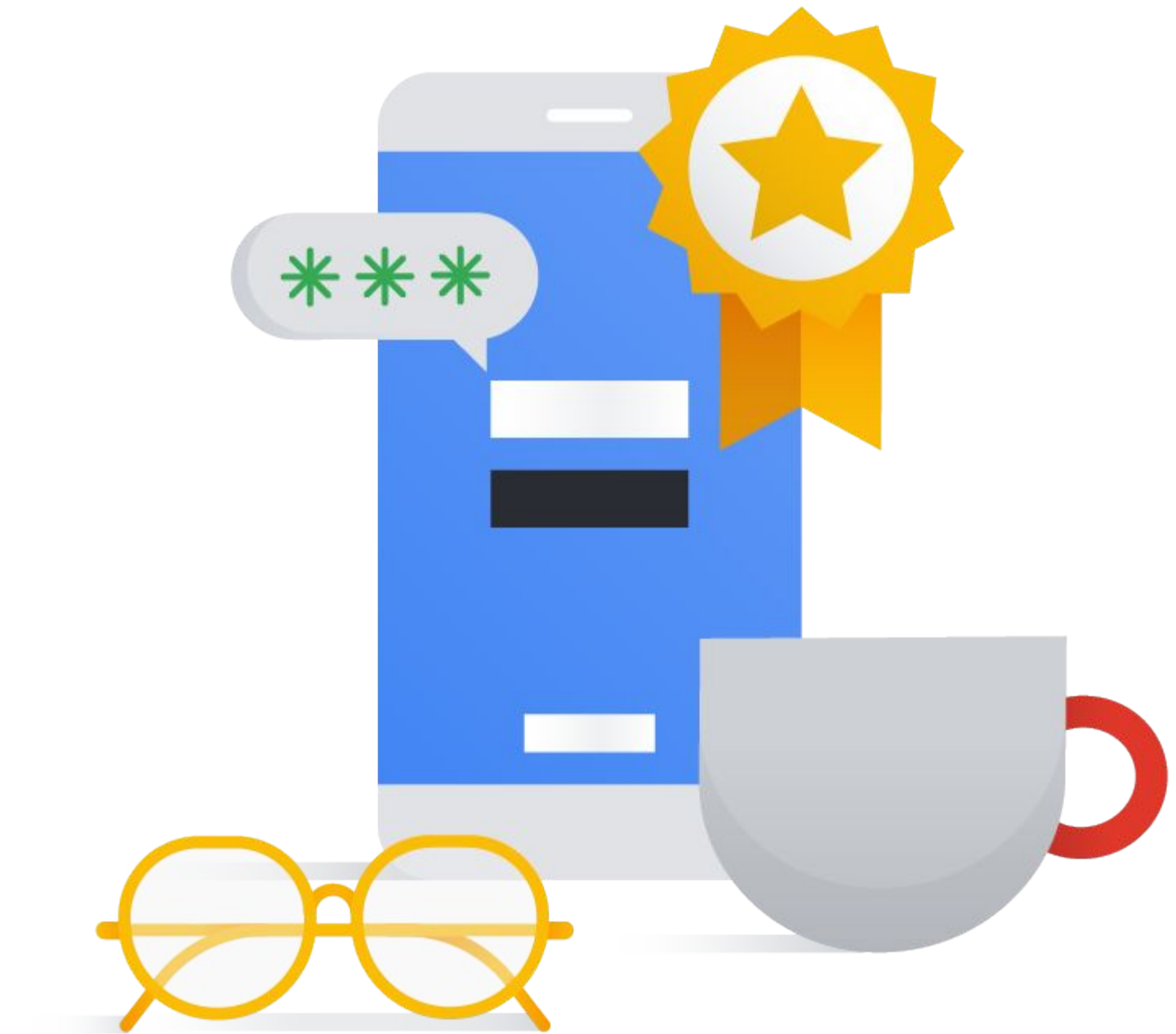
Use pytest to run your test files

```
def test_with_single_test_file():  
    """Test the agent's basic ability via a session file."""  
    AgentEvaluator.evaluate(  
        "tests.integration.fixture.home_automation_agent",  
        "tests/integration/fixture/home_automation_agent/simple_test.test.json",  
    )
```

Enable tracing by adding to your .env file

```
AF_TRACE_TO_CLOUD=1  
# It's expected that this will change to:  
# ADK_TRACE_TO_CLOUD=1
```

Now let's do a short
quiz to **check your
knowledge!**



Quiz | Question 01

Question

What are the two types of evaluation that ADK evaluation currently provides?

- A. The average response, and trajectory and tool use
- B. The final trajectory, and tool use
- C. The final response, and trajectory and tool use
- D. The final response, and function calling

Quiz | Question 01

Answer

What are the two types of evaluation that ADK evaluation currently provides?

- A. The average response, and trajectory and tool use
- B. The final trajectory, and tool use
- C. The final response, and trajectory and tool use
- D. The final response, and function calling



Quiz | Question 02

Question

What is a "trajectory" in the context of agent evaluation?

- A. The final output of the agent
- B. The user's initial query
- C. The evaluation score of the agent
- D. The series of actions an agent takes before responding

Quiz | Question 02

Answer

What is a "trajectory" in the context of agent evaluation?

- A. The final output of the agent
- B. The user's initial query
- C. The evaluation score of the agent
- D. The series of actions an agent takes before responding



Quiz | Question 03

Question

Which evaluation method involves creating individual test files for single interactions?

- A. Evalset approach
- B. Test file approach
- C. Web UI evaluation
- D. Command line evaluation

Quiz | Question 03

Answer

Which evaluation method involves creating individual test files for single interactions?

- A. Evalset approach
- B. Test file approach
- C. Web UI evaluation
- D. Command line evaluation



Google Cloud